Aim

To analyse the time and space efficiency of the ShopSphere e-commerce platform's back-end modules using exact operation counts, asymptotic notation, backward substitution, the Master Theorem, best/average/worst-case analysis, and a scalability audit for a catalogue of one million products.

Problem Statement

ShopSphere currently maintains 100,000 products and expects the catalogue to grow to 1,000,000 products. The platform must assess five modules: duplicate-listing detection (M1), API rate limiting (M2), product search (S1 and S2), recommendation generation (M4), and sales analytics (M5). The report generator has cost $f(n) = 3n^2 + 10n \log_2 n + 50$. The assigned question assumes that the server performs 10 basic operations per second and asks whether the modules can meet a nightly processing window of 10 minutes.

Tools and Requirements

- Language: Python 3
- External packages: None required; the program uses the Python standard library.
- Execution command: python scalability_audit.py
- Input sizes: n = 100,000 and n = 1,000,000 for scalability comparison; M2 is also verified for n = 1,024.
- Assumption for theoretical runtime estimates: 10 basic operations per second, as stated in the question.

Methodology

- Identify the basic operation for each module and derive its count or recurrence.

- Use exact summations for M1 and M2 and backward substitution for S2.
- Apply the Master Theorem to both M4 designs and use a recursion-tree/extended Master Theorem argument for M5.
- Compare best, average and worst cases for S1 and S2.
- Prove the report generator's $\Theta(n^2)$ bound using constants and disprove $O(n)$ using the formal definition.
- Evaluate operation counts at n = 1,000,000 and convert them to time using 10 operations per second.
- Execute the accompanying Python program to verify arithmetic and print a machine-readable summary.

Part (a) – Understand the Problem

Module	Input size	Dominant basic operation	Expected growth	Reason
M1 – Duplicate detector	n products	Compare the title and seller of a pair of products	$\Theta(n^2)$	Every product is compared with every later product.
M2 – API rate limiter	n requests	Audit(Request[j])	$\Theta(n)$	The inner-loop counts form a geometric series: 1 + 2 + 4 + … .
M3 – Product search S1	n products	Compare target with the current product ID	Best $\Theta(1)$; average/worst $\Theta(n)$	An unsorted list may require scanning many entries.
M3 – Product search S2	n sorted index entries	Compare target with the middle	$\Theta(\log n)$	Each comparison discards half the

		key		remaining index.
M4 – Recommendation engine	n customer-product records	Process recursive subproblems and merge	Current $\Theta(n^2)$; redesign $\Theta(n \log n)$	The current recurrence has four half-size calls; redesign has two.
M5 – Sales analytics	n sales records	Process/sort records at each recursive level	$\Theta(n \log^2 n)$	There are $\Theta(\log n)$ levels, each contributing $\Theta(n \log n)$.

Part (b) – Analyse the Iterative Modules

M1: Exact number of comparisons

For each i from 0 to n − 2, j runs from i + 1 to n − 1. Therefore, the number of comparisons for a fixed i is n − i − 1.

$C_1(n) = \Sigma_{i=0}^{n-2} (n - i - 1) = (n - 1) + (n - 2) + \ldots + 2 + 1.$

Using the sum of the first n − 1 positive integers: $C_1(n) = n(n - 1)/2 = (n^2 - n)/2$.

Thus, time complexity is $\Theta(n^2)$. The algorithm uses $\Theta(1)$ auxiliary space if the input catalogue is already stored and marking does not create a new data structure. If the list of duplicate indices is accumulated separately, that output storage depends on the number of duplicates.

M2: Exact number of audit operations

The values of i are 1, 2, 4, 8, …, 2^k, where $2^k < n$ and 2^k is the largest power of two below n. The inner loop performs i audits during an iteration.

Let $k = \lceil \log_2 n \rceil$ for n > 1. The loop values are 1, 2, …, 2^{k-1}, so $A(n) = \Sigma_{r=0}^{k-1} 2^r = 2^k - 1 = 2^{\lceil \log_2 n \rceil} - 1$.

Since $n \leq 2^{\lceil \log_2 n \rceil} < 2n$, the count is $\Theta(n)$, not $\Theta(n \log n)$. The team member's claim is incorrect because the inner-loop lengths do not each equal n; they form a geometric series.

For $n = 1{,}024 = 2^{10}$, the values are $1 + 2 + 4 + \ldots + 512 = 2^{10} - 1 = 1{,}023$ audit operations.

Space complexity is $\Theta(1)$ auxiliary space, since only the loop variables and current request are needed.

Part (c) – Solve S2 by Backward Substitution

The question gives $T(n) = T(n/2) + 1$ and $T(1) = 1$. Assume n is a power of two.

- $T(n) = T(n/2) + 1$
- $T(n/2) = T(n/4) + 1$, therefore $T(n) = T(n/4) + 2$
- $T(n) = T(n/8) + 3$
- After k substitutions: $T(n) = T(n/2^k) + k$
- Stop when $n/2^k = 1$, so $2^k = n$ and $k = \log_2 n$
- Using $T(1) = 1$: $T(n) = 1 + \log_2 n$

For $n = 2^k$, the exact recurrence solution is $T(n) = \log_2 n + 1$ comparisons. For arbitrary n, the comparison count is $\Theta(\log n)$, with the precise count depending on the tree shape and stopping condition.

Implementation	Time complexity	Auxiliary space
Iterative binary search	$\Theta(\log n)$	$\Theta(1)$
Recursive binary search	$\Theta(\log n)$	$\Theta(\log n)$ call stack

Part (d) – Apply the Master Theorem

M4 current design: T(n) = 4T(n/2) + n

Parameter	Value
a	4
b	2
f(n)	$n = \Theta(n)$
n^(log_b a)	$n^{(\log_2 4)} = n^2$
Comparison	$f(n) = O(n^{(2-1)}) = O(n)$
Master Theorem case	Case 1
Result	$T(n) = \Theta(n^2)$

The current design has quadratic growth because replicated partitions cause four recursive calls on half-sized inputs. Doubling n approximately quadruples the dominant work.

M4 proposed redesign: T(n) = 2T(n/2) + n

Parameter	Value
a	2
b	2
f(n)	$n = \Theta(n)$
n^(log_b a)	$n^{(\log_2 2)} = n$
Comparison	$f(n) = \Theta(n^{(\log_b a)})$
Master Theorem case	Case 2
Result	$T(n) = \Theta(n \log n)$

After redesign, doubling n increases the dominant work by approximately a factor of $2 \times \log_2(2n)/\log_2(n)$, which approaches 2 for large n. Removing replication improves the asymptotic growth from quadratic to n log n.

M5: T(n) = 2T(n/2) + n log n

Here a = 2, b = 2 and n^(log_b a) = n. The non-recursive work is f(n) = n log n, which is Θ(n log n), larger than n by a logarithmic factor. The three basic Master Theorem cases in their simplest form do not directly describe this logarithmic-factor extension; use the extended case or a recursion tree.

At each recursion-tree level, the subproblem sizes sum to n. At a level with subproblem size $n/2^r$, the combined non-recursive work is $n(\log_2 n - r)$. Summing for r = 0 through $\log_2 n - 1$ gives $n \sum_{s=1}^{\log 2n} s = \Theta(n(\log n)^2)$. Therefore, $T(n) = \Theta(n \log^2 n)$.

Part (e) – Best, Average and Worst Cases

Algorithm	Case	Input condition	Comparisons
S1 linear search	Best	Target is first item	1
S1 linear search	Average (successful, uniformly located)	Target position equally likely from 1 to n	(n + 1)/2
S1 linear search	Worst successful	Target is last item	n
S1 linear search	Unsuccessful	Target is absent	n
S2 binary search	Best	Target equals the first middle key	1
S2 binary search	Average	Depends on target distribution and tree shape	Θ(log n)
S2 binary search	Worst	Target is deepest leaf	Θ(log n)

		or absent after the last interval	

S1 expected comparisons when existence probability is p

Assume a successful target is equally likely to occur at any of the n positions, and an unsuccessful search checks all n items. Conditional expected comparisons are (n + 1)/2 when present and n when absent.

$E[C] = p(n + 1)/2 + (1 - p)n = n - p(n - 1)/2$.

For n = 1,000,000 and p = 0.5: $E[C] = 1{,}000{,}000 - 0.5 \times 999{,}999 / 2 = 750{,}000.25$ comparisons.

This result assumes uniform position for successful searches. If the question uses a different distribution, the expected count would need to be adjusted.

Part (f) – Prove the Asymptotic Bound

Given $f(n) = 3n^2 + 10n \log_2 n + 50$, show $f(n) = \Theta(n^2)$. For $n \geq 1$, $\log_2 n \leq n$, so $n \log_2 n \leq n^2$. Also, $50 \leq 50n^2$. Thus $f(n) \leq 3n^2 + 10n^2 + 50n^2 = 63n^2$.

For the lower bound, $f(n) \geq 3n^2$ for all $n \geq 1$. Choose $c_1 = 3$, $c_2 = 63$ and $n_0 = 1$. Then $3n^2 \leq f(n) \leq 63n^2$ for all $n \geq n_0$. Therefore $f(n) = \Theta(n^2)$.

Show f(n) is not O(n)

If f(n) were O(n), there would be constants $c > 0$ and n_0 such that $f(n) \leq cn$ for every $n \geq n_0$. But $f(n) \geq 3n^2$, so this would require $3n^2 \leq cn$, or $3n \leq c$, for arbitrarily large n. This is impossible for any fixed c. Therefore $f(n) \notin O(n)$.

Part (g) – Scalability Decision

The assignment states that the server performs 10 basic operations per second and the processing window is 10 minutes = 600 seconds. Therefore, the nominal capacity is 6,000 basic operations during the window. The estimates below count the dominant operations; actual wall-clock time can be greater because of constants and system overhead.

Module/design	**Operation count at n = 1,000,000**	**Estimated time at 10 ops/s**	**Within 10 minutes?**
M1	$n(n-1)/2$ = 499,999,500,000 comparisons	49,999,950,000 seconds ≈ 1,584.4 years	No
M2	$2^{\lceil \log_2 n \rceil} - 1$ = 1,048,575 audits	104,857.5 seconds ≈ 29.1 hours	No
S2	$\log_2 n + 1 \approx 20.93$; use about 20–21 comparisons	About 2.0–2.1 seconds	Yes
M4 current	$\Theta(n^2)$; leading-scale estimate $n^2 = 10^{12}$	About 100,000,000,000 seconds	No
M4 redesign	$\Theta(n \log_2 n)$; leading-scale estimate $n \log_2 n \approx 19{,}931{,}569$	About 1,993,157 seconds ≈ 23.1 days	No under this stated rate
M5	$\Theta(n \log_2^2 n)$; leading-scale estimate $n(\log_2 n)^2 \approx 398{,}631{,}371$	About 39,863,137 seconds ≈ 461.4 days	No

Important interpretation: the question provides recurrences and asymptotic forms but not all recurrence base-case costs or exact operation constants. For M4 and M5, these are leading-scale estimates rather than exact instruction counts. The 10-operations-per-second assumption is very restrictive; apply it consistently when answering the question.

Redesign Recommendations

- M1: Replace pairwise comparison with a hash-based grouping by (seller, normalized title), expected $\Theta(n)$, or sort by the key and scan adjacent records in $\Theta(n \log n)$.
- M2: The given doubling-window algorithm is $\Theta(n)$, but its exact count exceeds the stated 10-minute capacity at one million requests under the supplied rate. Consider batching, incremental counters, and suitable rate-limiting data structures.
- S2: Keep binary search on a sorted index, or use an appropriate database index for production-scale queries.
- M4: Remove replicated recursive work as proposed; this changes $\Theta(n^2)$ to $\Theta(n \log n)$, although the stated throughput assumption still predicts more than 10 minutes.
- M5: Review the repeated sorting/processing at each level. Seek a design with linear or n log n total work where the business requirements permit it, and profile actual implementation costs.
- Validate theoretical estimates with repeated execution on realistic data; distinguish asymptotic operation counts from measured wall-clock time.

Python Implementation and Verification

The accompanying source file scalability_audit.py calculates the principal operation-count estimates and checks the M2 count at n = 1,024. Execute it using Python and capture the terminal output for the

final report. This report document is a prepared solution draft; add your own actual source-code and execution screenshots before submission.

SOURCE CODE:

assessment.py - C:/Users/Arun/Downloads/assessment.py (3.11.9)

File Edit Format Run Options Window Help

```
"""
ShopSphere Scalability Audit
Course: CSA06 - Design and Analysis of Algorithms
Assessment: CO1 - Assessment Tool 1

This program calculates operation counts and scalability estimates
for the modules described in the assignment.
"""

from math import ceil, log2

# Assignment assumptions
OPS_PER_SECOND = 10
WINDOW_SECONDS = 10 * 60
CATALOGUE_SIZE = 1_000_000


# ------------------------------------------------
# M1: Duplicate-Listing Detector
# ------------------------------------------------

def m1_comparisons(n: int) -> int:
    """
    Exact number of product-pair comparisons:
    C(n) = n * (n - 1) / 2

    Time complexity: Theta(n^2)
    Auxiliary space: Theta(1)
    """
    if n < 0:
        raise ValueError("n must be non-negative")

    return n * (n - 1) // 2


# ------------------------------------------------
# M2: API Rate Limiter
# ------------------------------------------------

def m2_audits(n: int) -> int:
    """
    Counts audits performed by:

        i = 1
        while i < n:
            audit i requests
            i = i * 2

    Exact count for n > 1:
```

assessment.py - C:/Users/Arun/Downloads/assessment.py (3.11.9)

File Edit Format Run Options Window Help

```
        2^ceil(log2(n)) - 1

    Time complexity: Theta(n)
    Auxiliary space: Theta(1)
    """
    if n <= 1:
        return 0

    return (1 << ceil(log2(n))) - 1


# ------------------------------------------------
# M3: Product Search - Version S1
# Linear Search
# ------------------------------------------------

def linear_search(product_ids: list, target: int) -> int:
    """
    Searches sequentially through an unsorted list.
    Returns the index if found; otherwise returns -1.

    Best-case time: Theta(1)
    Average-case time: Theta(n), under the usual
                       uniform-position assumptions
    Worst-case time: Theta(n)
    Auxiliary space: Theta(1)
    """
    for index, product_id in enumerate(product_ids):
        if product_id == target:
            return index

    return -1


def s1_average_comparisons(n: int, p: float) -> float:
    """
    Expected comparisons for linear search.

    Assumptions:
    - Probability p that the target exists.
    - If present, each of the n positions is equally likely.
    - If absent, all n items are examined.

    E[C] = p(n + 1)/2 + (1 - p)n
         = n - p(n - 1)/2
    """
    if n < 1:
        raise ValueError("n must be at least 1")

    if not 0 <= p <= 1:
```

assessment.py - C:/Users/Arun/Downloads/assessment.py (3.11.9)

File Edit Format Run Options Window Help

```
        raise ValueError("p must be between 0 and 1")

    return n - p * (n - 1) / 2


# ------------------------------------------------
# M3: Product Search - Version S2
# Binary Search
# ------------------------------------------------

def binary_search(sorted_product_ids: list, target: int) -> int:
    """
    Searches a sorted list by repeatedly halving
    the search interval.

    Best-case time: Theta(1)
    Average-case time: Theta(log n)
    Worst-case time: Theta(log n)
    Auxiliary space: Theta(1), iterative version
    """
    low = 0
    high = len(sorted_product_ids) - 1

    while low <= high:
        mid = (low + high) // 2

        if sorted_product_ids[mid] == target:
            return mid

        elif sorted_product_ids[mid] < target:
            low = mid + 1

        else:
            high = mid - 1

    return -1


def s2_comparisons_power_of_two(n: int) -> int:
    """
    Solves T(n) = T(n/2) + 1, T(1) = 1
    for n being a positive power of two.

    Closed form: T(n) = log2(n) + 1
    """
    if n < 1 or n & (n - 1):
        raise ValueError(
            "n must be a positive power of two"
        )
```

```
    return int(log2(n)) + 1


# ------------------------------------------------
# M4: Recommendation Engine
# ------------------------------------------------

def m4_current_scale(n: int) -> int:
    """
    Current recurrence:
        T(n) = 4T(n/2) + n

    Complexity: Theta(n^2)

    Returns the leading-scale estimate n^2.
    This is not an exact recurrence operation count.
    """
    return n ** 2


def m4_redesign_scale(n: int) -> float:
    """
    Redesigned recurrence:
        T(n) = 2T(n/2) + n

    Complexity: Theta(n log n)

    Returns n * log2(n) as a leading-scale estimate.
    This is not an exact recurrence operation count.
    """
    if n <= 0:
        raise ValueError("n must be positive")

    return n * log2(n)


# ------------------------------------------------
# M5: Sales Analytics Engine
# ------------------------------------------------

def m5_scale(n: int) -> float:
    """
    Recurrence:
        T(n) = 2T(n/2) + n log n

    Complexity: Theta(n log^2 n)

    Returns n * (log2(n))^2 as a leading-scale
    estimate, not an exact recurrence operation count.
    """
```

File Edit Format Run Options Window Help

```
def m4_redesign_scale(n: int) -> float:
    """
    Redesigned recurrence:
        T(n) = 2T(n/2) + n

    Complexity: Theta(n log n)

    Returns n * log2(n) as a leading-scale estimate.
    This is not an exact recurrence operation count.
    """
    if n <= 0:
        raise ValueError("n must be positive")

    return n * log2(n)


# ------------------------------------------------
# M5: Sales Analytics Engine
# ------------------------------------------------

def m5_scale(n: int) -> float:
    """
    Recurrence:
        T(n) = 2T(n/2) + n log n

    Complexity: Theta(n log^2 n)

    Returns n * (log2(n))^2 as a leading-scale
    estimate, not an exact recurrence operation count.
    """
    if n <= 0:
        raise ValueError("n must be positive")

    return n * (log2(n) ** 2)


# ------------------------------------------------
# Runtime Estimation
# ------------------------------------------------

def estimated_seconds(operations: float) -> float:
    """
    Estimates runtime using the assignment assumption
    of 10 basic operations per second.
    """
    return operations / OPS_PER_SECOND


def format_duration(seconds: float) -> str:
    """Converts seconds into a readable duration."""
```

File Edit Format Run Options Window Help

```
    if seconds < 60:
        return f"{seconds:,.2f} seconds"

    if seconds < 3600:
        return f"{seconds / 60:,.2f} minutes"

    if seconds < 86400:
        return f"{seconds / 3600:,.2f} hours"

    return f"{seconds / 86400:,.2f} days"


# ------------------------------------------------
# Main Program
# ------------------------------------------------

def main() -> None:
    n = CATALOGUE_SIZE

    print("=" * 76)
    print("        SHOPSPHERE E-COMMERCE SCALABILITY AUDIT")
    print("=" * 76)

    print(f"Catalogue size: {n:,} products")
    print(f"Server throughput: {OPS_PER_SECOND} operations/second")

    capacity = OPS_PER_SECOND * WINDOW_SECONDS

    print(
        f"10-minute processing capacity: "
        f"{capacity:,} basic operations"
    )

    print("\n1. M1 - DUPLICATE-LISTING DETECTOR")
    print("-" * 76)

    m1_ops = m1_comparisons(n)

    print(f"Exact comparisons: {m1_ops:,}")
    print("Time complexity: Theta(n^2)")
    print("Auxiliary space: Theta(1)")
    print(
        "Estimated runtime:",
        format_duration(estimated_seconds(m1_ops))
    )

    print("\n2. M2 - API RATE LIMITER")
    print("-" * 76)

    m2_ops = m2_audits(n)
```

File Edit Format Run Options Window Help

```
    print(f"Exact audit operations: {m2_ops:,}")
    print("Time complexity: Theta(n)")
    print("Auxiliary space: Theta(1)")
    print(
        "Estimated runtime:",
        format_duration(estimated_seconds(m2_ops))
    )

    print("\nM2 VERIFICATION FOR n = 1,024")
    print("-" * 76)

    verification_n = 1024
    verification_ops = m2_audits(verification_n)

    print(f"Input size: {verification_n:,}")
    print(f"Calculated audits: {verification_ops:,}")
    print("Expected audits: 1,023")

    assert verification_ops == 1023

    print("M2 verification: PASSED")

    print("\n3. M3 - PRODUCT SEARCH")
    print("-" * 76)

    print("S1 - Linear Search")
    print("Best-case comparisons: 1")
    print(f"Worst-case comparisons: {n:,}")
    print("Time complexity: Theta(n)")
    print("Auxiliary space: Theta(1)")

    p = 0.5
    average_ops = s1_average_comparisons(n, p)

    print(f"\nExpected comparisons when p = {p}:")
    print(f"{average_ops:,.2f}")

    print("\nS2 - Binary Search")
    print("Time complexity: Theta(log n)")
    print("Iterative auxiliary space: Theta(1)")
    print("Recursive auxiliary space: Theta(log n)")
    print(
        "Approximate comparisons for one million items:",
        ceil(log2(n))
    )

    print("\n4. M4 - RECOMMENDATION ENGINE")
    print("-" * 76)
```

File Edit Format Run Options Window Help

```
    print(f"Maximum operations in 10 minutes: {capacity:,}\n")
    print(f"{'Module':<18}{'Operations':>24}{'Decision':>22}")
    print("-" * 64)

    for module, operations in results:
        decision = (
            "WITHIN LIMIT"
            if operations <= capacity
            else "EXCEEDS LIMIT"
        )

        print(
            f"{module:<18}{operations:>24,.2f}"
            f"{decision:>22}"
        )

    print("\n7. VERIFICATION CHECKS")
    print("-" * 76)

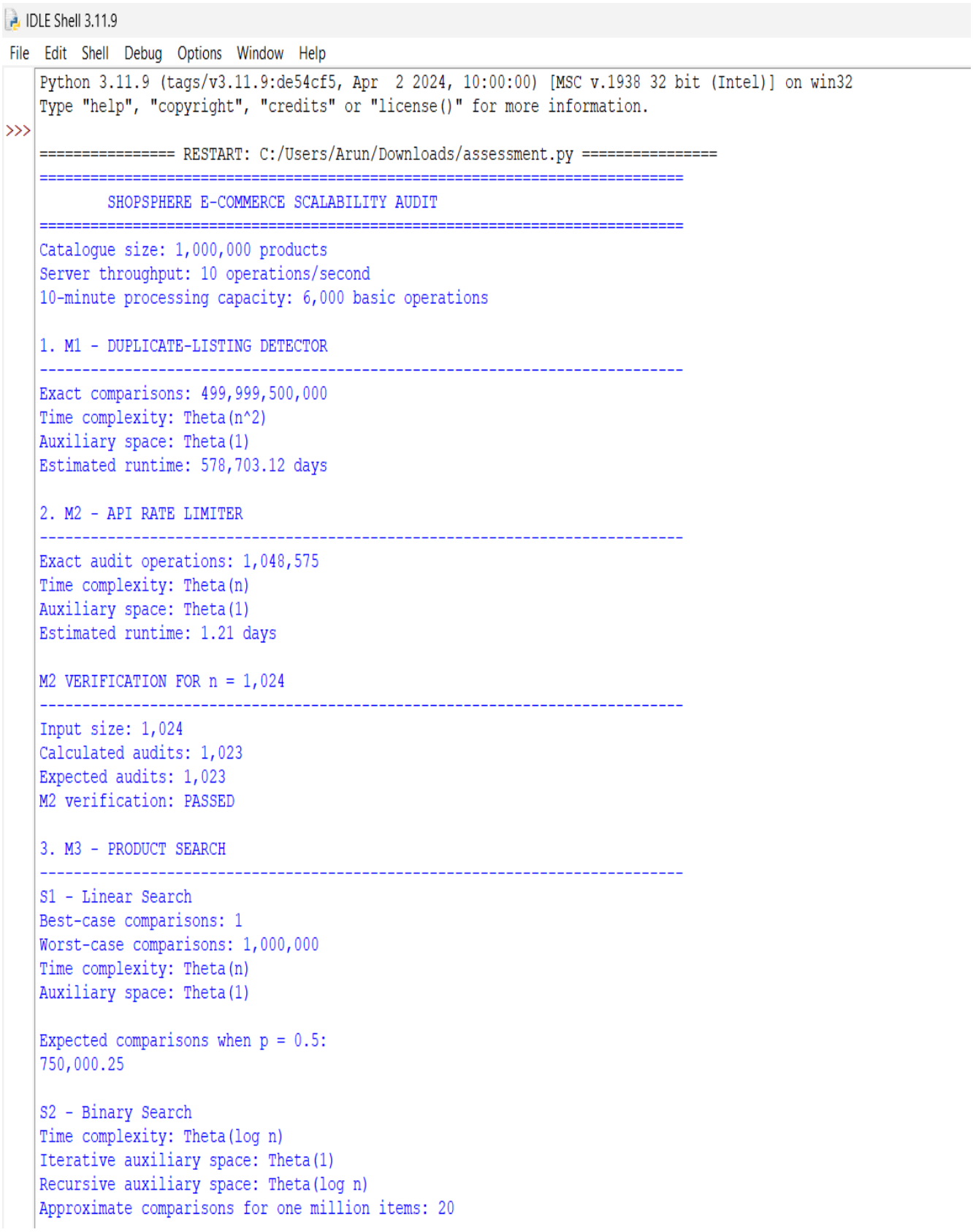
    assert m1_comparisons(5) == 10
    assert m2_audits(1024) == 1023
    assert linear_search([10, 20, 30], 20) == 1
    assert binary_search([10, 20, 30], 20) == 1
    assert abs(
        s1_average_comparisons(1_000_000, 0.5)
        - 750_000.25
    ) < 1e-9

    print("M1 operation-count test: PASSED")
    print("M2 operation-count test: PASSED")
    print("Linear search test: PASSED")
    print("Binary search test: PASSED")
    print("Average-comparison test: PASSED")

    print("\n" + "=" * 76)
    print("AUDIT COMPLETED SUCCESSFULLY")
    print(
        "Note: M4 and M5 operation counts are leading-scale "
        "estimates, not exact counts."
    )
    print(
        "The runtime estimates use the assignment's simplified "
        "10 operations/second assumption."
    )
    print("=" * 76)


if __name__ == "__main__":
    main()
```

OUTPUT SCREENSHOT:



```
4. M4 - RECOMMENDATION ENGINE
--------------------------------------------------------------------------------
Current design: T(n) = 4T(n/2) + n
Complexity: Theta(n^2)
Leading-scale estimate: 1,000,000,000,000
Estimated runtime: 1,157,407.41 days

Proposed redesign: T(n) = 2T(n/2) + n
Complexity: Theta(n log n)
Leading-scale estimate: 19,931,569
Estimated runtime: 23.07 days

5. M5 - SALES ANALYTICS ENGINE
--------------------------------------------------------------------------------
Recurrence: T(n) = 2T(n/2) + n log n
Complexity: Theta(n log^2 n)
Leading-scale estimate: 397,267,426
Estimated runtime: 459.80 days

6. NIGHTLY PROCESSING DECISION
--------------------------------------------------------------------------------
Maximum operations in 10 minutes: 6,000

Module                          Operations              Decision
------------------------------------------------------------------
M1                      499,999,500,000.00         EXCEEDS LIMIT
M2                           1,048,575.00         EXCEEDS LIMIT
S2                                 20.00          WITHIN LIMIT
M4 current            1,000,000,000,000.00         EXCEEDS LIMIT
M4 redesign                  19,931,568.57         EXCEEDS LIMIT
M5                          397,267,425.63         EXCEEDS LIMIT

7. VERIFICATION CHECKS
--------------------------------------------------------------------------------
M1 operation-count test: PASSED
M2 operation-count test: PASSED
Linear search test: PASSED
Binary search test: PASSED
Average-comparison test: PASSED

================================================================================
AUDIT COMPLETED SUCCESSFULLY
Note: M4 and M5 operation counts are leading-scale estimates, not exact counts.
The runtime estimates use the assignment's simplified 10 operations/second assumption.
================================================================================
>>>
```

Analysis

The ShopSphere Scalability Audit analyses the time and space complexity of five modules as the product catalogue grows from 100,000 to 1,000,000 products.

- M1 – Duplicate-Listing Detector: Performs (n(n-1)/2) comparisons, giving (\Theta(n^2)) time complexity.

- M2 – API Rate Limiter: Performs (2^{\lceil\log_2 n\rceil}-1) audits, giving (\Theta(n)) time complexity.

- M3 – Product Search: Linear Search has (\Theta(n)) average and worst-case complexity, while Binary Search has (\Theta(\log n)) time complexity.

- M4 – Recommendation Engine: The current design has (\Theta(n^2)) complexity. The proposed redesign improves it to (\Theta(n\log n)).

- M5 – Sales Analytics Engine: The recurrence (T(n)=2T(n/2)+n\log n) gives (\Theta(n\log^2 n)) complexity.

- Asymptotic Proof: The report generator function (f(n)=3n^2+10n\log n+50) is (\Theta(n^2)) and is not (O(n)).

- The scalability estimates identify which modules exceed the 10-minute processing limit under the assignment's assumption of 10 basic operations per second. The results help identify inefficient algorithms and recommend improvements for handling a growing product catalogue.

Conclusion

The audit shows that algorithm design determines whether ShopSphere can scale. M1 is quadratic and should be redesigned. M2 has a geometric-series operation count and is $\Theta(n)$, not $\Theta(n \log n)$, but its count still exceeds the stated capacity at n = 1,000,000. S2 is logarithmic and scales well. M4 improves from $\Theta(n^2)$ to $\Theta(n \log n)$ by removing replicated recursion. M5 is $\Theta(n \log^2 n)$. The report generator is $\Theta(n^2)$, and its cost is not $O(n)$. Theoretical results should be complemented by execution evidence and realistic profiling.